

PRÉ-RENTREE 2026-2027

SÉANCE 5 — RÉINVESTIR, CONSOLIDER,
PRÉPARER LA RENTREE

Livret de travail — Ahmed BENHADJ SALEM

Projet autonome — parcours candidat individuel

Entrée en Première — Numérique et sciences informatiques — NSI — 1 h 15 de travail, puis 45 minutes d'évaluation dans un document séparé

Objectif personnel de cette séance. Fiabiliser les accumulateurs — erreur documentée dans les deux bilans — et installer une pratique de test systématique, compétence la plus basse du profil, décisive pour un parcours en autonomie.

Ce livret couvre uniquement les 75 premières minutes. L'évaluation finale est remise ensuite, dans un document distinct.

Le fil des cinq séances

- S1 Variables, affectation et conditions ;
- S2 Boucles, compteurs et accumulateurs ;
- S3 Fonctions, contrats et tests ;
- S4 Listes, enregistrements et données CSV ;
- S5 aujourd'hui : réactiver, consolider deux axes personnels, faire le pont vers la Première NSI, puis mesurer.

Feuille de route des 75 minutes

Temps	Travail	Preuve attendue
0–10 min	Réactiver sans ordinateur les cinq constructions travaillées de la séance 1 à la séance 4.	Cinq réponses écrites et le contrôle utilisé.
10–35 min	Accumulateurs : somme, compteur, invariant et variant	Trois réussites autonomes sur l'axe prioritaire.
35–55 min	Tests, assertions et cas limites	Deux réussites autonomes sur le second axe.
55–70 min	Chercher dans une liste : du parcours complet à la dichotomie	Une production reliant un acquis de Seconde (SNT et algorithmique) à un attendu de Première NSI.
70–75 min	Cadrage méthodologique, sans aucune information sur le contenu de l'évaluation.	Deux engagements écrits avant l'évaluation.

Règle de la séance. J'écris ma réponse *et* le contrôle qui la rend défendable. Une réponse sans contrôle reste une hypothèse. La ligne « Certitude » se remplit *avant* toute vérification.

Phase 1 — Réactivation

0–10 min

Réactiver sans ordinateur les cinq constructions travaillées de la séance 1 à la séance 4.

Sept minutes sans machine. L'ordinateur ne sert ensuite qu'à *contrôler*, jamais à trouver la réponse.

1. Pour $L = [5, 2, 9, 4]$, donner $L[0]$, $L[-1]$, $\text{len}(L)$ et le dernier indice valide.

2. Écrire la négation de $(x \geq 3)$ and $(x \leq 12)$.

3. Donner les valeurs produites par `range(2, 11, 3)` et le nombre de tours de boucle.

4. Après `a = 4` puis `b = a` puis `a = 9`, que valent `a` et `b` ?

5. Que renvoie une fonction qui ne contient aucune instruction `return` ? Que vaut alors la variable qui reçoit le résultat ?

Certitude (1 à 4) : ☐ 1 ☐ 2 ☐ 3 ☐ 4

Le contrôle que j'ai utilisé : _____

Aide maximale utilisée : ☐ aucune ☐ comprendre ☐ identifier ☐ choisir la méthode ☐ démarrer

Phase 2 — Consolidation, axe prioritaire

10–35 min

Accumulateurs : somme, compteur, invariant et variant

Deux variables, deux rôles distincts. Un **accumulateur** de somme cumule des valeurs ; un **compteur** compte des tours, indépendamment du contenu. Tous deux s'initialisent **avant** la boucle, jamais à l'intérieur.

Invariant : propriété vraie avant et après chaque tour, par exemple « `total` contient la somme des valeurs déjà parcourues ». **Variant** : quantité entière positive qui décroît et garantit l'arrêt.

Exemple guidé. Somme des valeurs de `range(1, 4)`.

```
total = 0
for i in range(1, 4):
    total = total + i
```

1. Les valeurs parcourues sont 1, 2, 3 : trois tours.
2. États successifs de `total` : 1, puis 3, puis 6.
3. Le résultat est 6, et non 3 : 3 est le *nombre* de tours, pas la somme.
4. Invariant : après k tours, `total` contient la somme des k premières valeurs.

À toi

1. Tracer l'exécution de `total = 0` puis `for i in range(1, 6): total = total + i`, en écrivant l'état après chaque tour, puis donner le résultat.

2. Écrire une fonction `bilan(L)` qui renvoie la somme des éléments de `L` *et* le nombre de valeurs strictement négatives.

3. Énoncer l'invariant de la boucle de l'exercice 1, puis indiquer ce qui joue le rôle de variant.

4. Trouver l'erreur et corriger : `for v in L:` suivi de `total = 0` puis `total = total + v`.

5. Écrire une boucle qui calcule le maximum d'une liste non vide, et justifier l'initialisation choisie.

6. Un élève annonce que la somme des valeurs de `range(1, 4)` vaut 3. Expliquer précisément la confusion, puis donner la valeur exacte.

Critère de réussite de cette phase : deux accumulateurs corrects écrits en autonomie, avec un invariant énoncé et un tableau d'état exact.

Certitude (1 à 4) : ☐1 ☐2 ☐3 ☐4

Le contrôle que j'ai utilisé : _____

Phase 3 — Consolidation, second axe

35–55 min

Tests, assertions et cas limites

Un test, c'est une entrée et un résultat attendu, écrits avant l'exécution. Une **assertion** vérifie ce couple : `assert f(4) == 16`.

Cas limites à examiner systématiquement : structure vide, un seul élément, valeur absente, première et dernière position, valeur nulle ou négative. **Règle :** un test qui échoue n'est pas effacé ; il localise le premier écart.

À toi

1. Pour une fonction `maximum(L)`, écrire quatre tests avec leur résultat attendu, dont deux cas limites.

2. Écrire en Python les assertions correspondant aux trois premiers tests de l'exercice précédent.

3. L'assertion `assert 0.1 + 0.2 == 0.3` échoue. Expliquer pourquoi, puis proposer une écriture correcte.

4. Spécifier la fonction `indice_de(L, cible)` : entrées, sortie attendue, comportement lorsque la cible est absente.

Critère de réussite de cette phase : un jeu de tests couvrant au moins un cas ordinaire et deux cas limites, écrit avant toute exécution.

Certitude (1 à 4) : ☐1 ☐2 ☐3 ☐4

Le contrôle que j'ai utilisé : _____

Phase 4 — Réinvestissement

55–70 min

Découverte guidée : construire, à partir du parcours de liste déjà travaillé, deux algorithmes nouveaux du programme de Première — la recherche séquentielle et la recherche dichotomique, avec leur précondition et leur coût.

Ce qui change en Première NSI. Un parcours de liste n'est plus une fin en soi : il devient un **algorithme**, c'est-à-dire une procédure dont on sait dire ce qu'elle renvoie, dans quels cas, et à quel coût.

Ce qui est réinvesti : la boucle, l'indexation, le test et le contrat d'une fonction, travaillés de la séance 1 à la séance 4. **Ce qui est nouveau :** les **algorithmes de recherche** eux-mêmes, leur précondition et leur coût, qui appartiennent au programme de Première.

Cette phase introduit des notions de l'année à venir. Ce que tu produis ici ne sera donc *pas* comparé à ton positionnement de début de stage : il ne s'agit pas de mesurer un progrès, mais de préparer les premières séances de septembre.

A. La recherche séquentielle — 8 min

```
def indice_de(L, cible):
    for i in range(len(L)):
        if _____:
            return ____
    return -1
```

1. Compléter les deux trous. Pourquoi renvoie-t-on -1 et non 0 lorsque la cible est absente ?

2. Tracer l'exécution pour $L = [7, 3, 9, 3]$ et $cible = 3$.

Tour	i	L[i]	Condition vraie ?
1			
2			

3. Que renvoie l'appel ? _____ Et `indice_de([7, 3, 9, 3], 5)` ? _____
4. Combien de comparaisons au maximum pour une liste de 100 éléments ? _____

B. Et si la liste était triée ? — 7 min

On cherche 41 dans la liste triée $[3, 8, 12, 19, 25, 33, 41, 50]$, dont les indices vont de 0 à 7.

Convention retenue pour tout l'exercice : la zone de recherche est délimitée par deux indices g et d ; l'indice du milieu est $m = (g + d) // 2$, c'est-à-dire le quotient entier. Une autre convention donnerait un autre décompte : c'est pourquoi on la fixe avant de commencer.

1. Premier tour : $g = 0$ et $d = 7$. Calculer m , comparer $L[m]$ à 41, puis dire quelle moitié est éliminée.
2. Poursuivre en notant g , d et m à chaque tour. Combien de comparaisons ont été nécessaires ? _____
3. Compléter : à chaque étape, la zone de recherche est divisée par _____.
4. Écrire la **précondition** indispensable à cet algorithme. _____
5. Un élève applique cette méthode à une liste non triée et trouve « absent » alors que la valeur y figure. Expliquer précisément l'origine de l'erreur.

Preuve attendue : la valeur de retour en cas d'absence est justifiée ; la précondition de tri est énoncée avant tout jugement sur l'efficacité.

Phase 5 — Avant l'évaluation

70–75 min

Cadrage méthodologique. Aucun contenu de l'évaluation n'est annoncé.

Cinq règles, et rien d'autre. Ce cadrage ne porte sur aucun contenu de l'évaluation.

1. **Gérer le temps.** Trois parties : environ 10 min, 20 min et 15 min. Un regard sur l'horloge à la fin de chaque partie suffit.
2. **Justifier.** Écrire la relation, la propriété ou la règle avant le calcul. Un résultat seul ne montre pas ce que tu sais.
3. **Contrôler.** Avant de passer à la suite : ordre de grandeur, substitution, unité, ou vérification par une seconde voie.
4. **Lire jusqu'au bout.** Souligner la question réellement posée et les données effectivement fournies.
5. **Passer et revenir.** Une question laissée de côté puis reprise vaut mieux qu'une réponse écrite au hasard.

La certitude. Elle est demandée à trois moments seulement, comme au test de positionnement de début de stage : ☐1 ☐2 ☐3 ☐4 . Elle n'entre pas dans la note. Elle sert à distinguer ce que tu sais de ce que tu crois savoir.

Mon engagement d'avant-évaluation

Le contrôle que j'écrirai systématiquement : _____

La question que je traiterai en dernier : _____

Fin du livret de travail. L'évaluation finale est un document séparé, remis par l'enseignant à l'issue de ces 75 minutes.